



Den

A quiet place to play the games you already own. Drop a pile of downloads on it; it sorts, repairs, and shelves them. Pick up any controller; it just works. Pull the plug mid-boss; you resume where you stood. The fourth sibling of **Lockbox**, **Hearth**, and **lanflix** — different job, same interface.

TAURI V2

DEBIAN X86-64

RASPBERRY PI (ARM64)

WINDOWS 10/11

LOCAL-FIRST

NO TELEMETRY

01 · WHAT IT IS

A launcher, not another emulator

Den is a Tauri v2 desktop app — Rust backend, vanilla-JavaScript frontend, no framework — exactly like its siblings. It does three jobs and refuses all others:

- **Intake.** A drop zone that accepts whatever a real download folder contains — zips inside zips, broken archives, stray manuals, five regional copies of the same game — and turns it into a clean, named, playable library. Every file is accounted for with a word, never silently eaten.
- **Play.** One click (or one controller press) boots the game through a proven emulation runtime. Den orchestrates *libretro* cores via a managed RetroArch runner rather than reimplementing twenty consoles — that is what keeps it lightweight and what makes the Raspberry Pi target realistic.
- **Continue.** Battery saves and save states are Den's responsibility, not the emulator's mood. Autosave every 60 seconds and on exit; power loss costs at most a minute. The library's front page leads with "Continue where you left off," thumbnail and all.

It never downloads games, and it ships no BIOS or copyrighted data. You bring the files; Den brings the order.

Your games, mapped to hardware

The titles you listed fall into three honest tiers. Den is designed around tier one, supports tier two where the machine can carry it, and tells you plainly about tier three instead of pretending.

TIER	EXAMPLES FROM YOUR LIST	SYSTEM	DEBIAN X64
Core libretro, built-in	Sonic, Earthworm Jim, General Chaos (Genesis) · Pocky & Rocky, Civilization (SNES) · Twisted Metal, Tekken 1–3, Soviet Strike, GTA 1–2 (PS1) · GoldenEye 007 (N64) · Aliens (arcade) · Civ I/II (DOS)	NES · SNES · Genesis/Sega CD/32X · N64 · PS1 · GB/GBC/GBA · Arcade (FBNeo) · DOS	runs
Heavy external emulator profiles	Tekken 4–5, GTA III– San Andreas, classic Call of Duty (PS2) · GameCube titles	PS2 (PCSX2) · GameCube/Wii (Dolphin)	runs
Native PC out of scope	Borderlands, modern Call of Duty, modern GTA, Civ V/VI	These are PC games, not ROMs — there belong to Steam/Proton/GOG. Den can lis later (a stretch goal), but will never claim will.	

Default core picks, chosen for accuracy-per-watt so the same library works on a Pi: `snes9x` , `genesis_plus_gx` , `picodrive` (32X), `mesen` / `fceumm` , `mupen64plus_next` , `swanstation` on x64 / `pcsx_rearmed` on Pi, `mgba` , `gambatte` , `fbneo` , `dosbox_pure` . Per-game core override lives in the game's page, never in a global maze.

Rust workspace, thin webview, isolated player

The Lockbox pattern, not the monolith: a workspace of small library crates that build headless in CI, plus a Tauri shell that only wires them to the window.

```
den/
├─ crates/
│  ├─ den-intake      unpack, identify, repair, file, report  (pure, fuzzable)
│  ├─ den-db          SQLite (WAL) library: games, hashes, saves, sessions
│  ├─ den-ident       DAT index (No-Intro / Redump), magic bytes, hashing
│  ├─ den-runner      RetroArch process control, config generation, watchdog
│  ├─ den-input       controller detection (gilrs + SDL gamecontrollerdb)
│  └─ den-core        glue the shell talks to
├─ apps/desktop/     Tauri v2 shell – vanilla JS/TS, Vite, no framework
│  └─ styles/         tokens.css · animations.css · base.css  (copied verbatim)
├─ corpus/           fixture archives for intake tests, broken ones included
├─ tools/brand.py    den mark added to the shared 24×24 brand grid
└─ docs/DESIGN.md    byte-identical copy, per family policy
```

Why a managed RetroArch runner instead of in-process libretro: video, audio, and input plumbing per platform (KMS/GLES on the Pi, D3D on Windows) is most of an emulator frontend's difficulty, and RetroArch has spent a decade on it. Den generates a private `retroarch.cfg` per session — its own save/state paths, autosave interval, controller config, quit hotkey — launches fullscreen, and watches the process. A core crash never takes the library down, and the last autosave survives. If a future need justifies it, an in-process libretro mode can be added behind the same `den-runner` interface without touching the UI.

RetroArch and cores are resolved per platform: apt packages on Debian/Pi where available, a bundled portable build on Windows, and an opt-in, hash-verified core download for anything missing. First-run setup says exactly what it fetched and from where; nothing downloads silently.

Drop anything; every file gets a word

Intake is the feature. It is built as a pure pipeline in `den-intake`, tested against a corpus of deliberately awful fixtures, and it treats your originals as read-only until the moment a copy is safely shelved.

1

Stage

Copy the drop into a staging area. Originals are never touched, moved, or deleted.

2

Unpack

zip, 7z, rar, tar, gz — nested to any depth. A corrupt archive is salvaged entry-by-entry: what extracts cleanly continues, what doesn't is quarantined with the reason.

3

Identify

Extension, then magic bytes, then CRC32/SHA-1 against a bundled No-Intro/Redump index. A hash match names the game exactly; a header-only match is shelved as "probable" and flagged for a look.

4

Repair

Regenerate a missing .cue for a lone .bin; build .m3u playlists for multi-disc games; keep headered and headerless ROMs straight; fix extensions that lie about their contents; optionally compress discs to CHD to reclaim space.

5

Shelve

Canonical names into `library/<system>/<Game>/`. Five regional copies dedupe by hash — best revision shown, variants kept but tucked away. BIOS files are recognized by hash and filed into the system folder automatically. Manuals, art, and readmes ride along under `_extras`. OS cruft is dropped.

6

Report

An intake report card lists every input file with its outcome as a word — never a colour, never silence.

The report vocabulary, in house style: `added` `duplicate` `repaired` `probable` `bios` `extra` `quarantined` `unsupported`

Edge cases the corpus must cover before v1

- Zip-in-zip-in-zip; one zip holding games for three different consoles; split archives with a part missing.
- Truncated downloads and CRC-failing entries — salvage the rest, quarantine the broken, say which and why.
- `.bin` without `.cue`, multi-disc PS1 sets, audio tracks, wrong or missing extensions, headered NES/SNES dumps.
- Rider files: manuals, scans, `.txt`, `.docx`, empty folders, desktop.ini/.DS_Store, and genuinely alien formats (a `.zim` wiki dump gets `unsupported`, not a crash).
- Someone else's save files in the pile — `.srm` battery saves are imported and attached to their game, not discarded. That's a gift, not junk.
- Password-protected archives — reported as such, with a place to type the password and retry.

05 · CONTROLLERS

Plug in anything shaped like a gamepad

- **Recognition.** `gilrs` plus the community SDL `gamecontrollerdb.txt` (thousands of devices, including the Amazon-special clones) identifies the pad and its physical layout the moment it's plugged in. Toast, house style: *"controller connected — 8BitDo SN30 Pro, player 1."* Hot-plug and multi-player included.
- **In-game.** Den writes RetroArch autoconfig from the same database, so the pad that navigated the library is the pad that plays, with no second setup. N64-shaped pads map natively onto the N64 core; a Genesis-style six-button pad lands on Street Fighter layouts sensibly; PlayStation and Xbox layouts both work without caring which label is on which button.
- **Positional, not lettered.** The UI never says "press A" — on a Nintendo layout that's the opposite button from an Xbox layout. Prompts use position and glyph ("right face button"), drawn per detected layout.

- **Remapping.** A press-to-bind screen with the house's sunken-pip test panel: press a button, watch its pip press in. Profiles are saved per controller and per system, stored in the library DB, exportable.
- **Couch mode.** The whole library is navigable by d-pad alone — a large-type, TV-distance view. On the Pi this is the primary mode, and an optional systemd unit (the `hearth.service` precedent) boots the machine straight into it.

06 · PERSISTENCE

The power cord is not a save system — but it may as well be

- **Battery saves** (`.srm`) live in the library, owned by Den, written where the game expects them.
- **Save states:** autosave every 60 seconds and on every exit. Launching a game with a newer autosave than its battery save asks one question — *Continue from 2 minutes ago, or start from the last in-game save?* — with Continue as the default. Worst case after a yanked cord: one minute lost.
- **Atomicity.** States are written to a temp file and renamed; the library DB runs SQLite in WAL mode; a crash mid-write can never corrupt the previous good state.
- **Sessions.** Play time and last-played are logged, which is all the front page needs: Continue, Recent, then the shelves.

07 · INTERFACE

The family look, shelf by shelf

Den copies the shared system verbatim rather than imitating it: `tokens.css` , `animations.css` , the control primitives in `base.css` , the six Gilroy/Avenir font files, the `createElementNS` icon table, and `docs/DESIGN.md` byte-for-byte, with the den mark added to `tools/brand.py` on the same 24×24, 2.0-stroke grid — proposed: *a CRT television holding a play triangle*, a container holding one emblem like the box-and-keyhole and the arch-and-flame.

The greyscale rule gets its best showcase here: the chrome has no accent colour anywhere, so the only colour on screen is the box art itself, sitting on flat paper like prints in a frame shop. Intake status is words in sunken wells; a long operation breathes instead of spinning; emphasis is inversion.

SCREEN	WHAT IT IS
Library	Raised sidebar island of systems; flat shelf of box art with Continue and Recent rows first. Search filters as you type; everything reachable by d-pad.
Game	Art, play button (inverted — the one solid thing on the page), save states with thumbnails, per-game core and controller overrides, the game's <code>_extras</code> .
Intake	The drop target (a sunken well the size of the window) and the report card. Quarantine lives here, with reasons and a retry.
Controllers	Connected pads as raised cards, press-to-test pips, remap flow.
Couch	TV-distance, controller-first rendering of the same library. Default on the Pi.

Box art comes from the libretro-thumbnails set, fetched hash-named, **opt-in only** — the family rule is that nothing talks to the network without saying so, and Den works fully (title text on a quiet tile) with art fetching off.

08 · PACKAGING

Three targets, told honestly

TARGET	PACKAGE	NOTES
Debian 12/13 x86-64	.deb + Applmage	Needs <code>libwebkit2gtk-4.1</code> ; RetroArch from apt or the opt-in fetch. Primary development platform.
Raspberry Pi	.deb (arm64)	Raspberry Pi OS Bookworm 64-bit. Pi 4 (4GB+) is the floor; Pi 5 recommended, and required for comfortable N64. <code>pcsx_rearmed</code> and friends chosen for ARM. Optional boot-to-couch-mode systemd unit.
Windows 10/11	NSIS installer	WebView2 evergreen bootstrapper; portable RetroArch bundled beside the app so nothing else needs installing.

Per the family's honesty-in-docs habit: each release states which targets were actually exercised, not just configured.

09 · MILESTONES

Build order, each step provable

MILESTONE	SCOPE	DONE MEANS
M0 · Hearthstone	Scaffold: Tauri v2 shell, design-system port, brand mark, empty library, CI (fmt, clippy, tests, axe a11y run).	App opens on all three targets; both themes correct; a11y pass on the built frontend.
M1 · Intake	<code>den-intake</code> + <code>den-ident</code> + corpus (40+ fixtures, broken ones included), report card UI.	Drop a messy multi-console zip pile → correctly shelved library; every input file accounted for with the right word; corpus green in CI.
M2 · Play	<code>den-runner</code> , core setup flow, saves/states, Continue.	Click Sonic → playing in under 5 seconds on all targets. Kill the power mid-game → relaunch resumes within 60 seconds of where you were.
M3 · Controllers	<code>den-input</code> , hot-plug, remap UI, couch mode.	Four differently-shaped pads (Nintendo, PlayStation, Xbox, N64 layouts) each work with zero setup, in menu and in game; full library run with d-pad only.
M4 · Shelves	Art fetch (opt-in), search, playtime, external-emulator profiles for the PS2/GC tier on x64/Windows, packaging polish.	Installable packages for all three targets; release notes state exercised platforms.

Decided up front

RISK	RULING
N64 on a Pi	GoldenEye is one of the heaviest N64 titles. Ship it honestly: Pi 5 at original resolution runs it; Pi 4 gets a "this game is demanding on this device" word on the game page, not a crash.
Webview on a Pi	Tauri's webkit2gtk footprint (~150 MB RAM) is fine for a launcher; the game itself runs outside the webview at native speed. Couch mode keeps DOM small.
RAR archives	Supported via the freeware unrar library (extraction-only license is compatible with this use). 7z and zip handled by pure-Rust crates.
BIOS files	Never shipped, never downloaded. Recognized by hash during intake and filed automatically; a PS1 game without its BIOS shows exactly which file is missing.
Legality	Den has no downloader, no store, no links to ROM sites. It organizes files the user provides. Same posture as its siblings: local-first, no telemetry, nothing leaves the machine except the opt-in art/core fetch it announces.
Name	Proposed: Den. One warm domestic syllable beside Hearth and Lockbox; the room where the TV and the controllers live. Runners-up if taken or disliked: Rumpus, Marquee, Attic.

Non-goals, so scope stays honest

- No netplay, no achievements, no accounts, no cloud sync in v1.
- No shader-pack browser or emulator settings labyrinth — sensible defaults per core, a per-game override, nothing else.
- No scraping metadata from the open web; identification is by hash against bundled databases, art is one opt-in fetch.

- No promise that modern PC games run under emulation. They don't, anywhere, and Den will not pretend.

Den · build plan · drafted 2026-08-19 · design system inherited from the shared Hearth/Lockbox `DESIGN.md`